

UNIT-V

Agile Methods with RUP and
PRINCE2

Agile Methods with RUP and PRINCE2
and Tools and Obstacles:

Agile Modelling and RUP

FDD and RUP

Agile Methods and PRINCE2

Tools to Help with Agile Development,
Eclipse: An Agile IDE.

Introduction

- we will focus on the Unified Process (also known as the Rational Unified Process or RUP) and PRINCE2.
- The Unified Process is a design framework that guides the tasks, people and products of the design process. It is a framework because it provides the inputs and outputs of each activity, but does not restrict how each activity must be performed. Different activities can be used in different situations, some being left out, others being replaced or augmented.
- PRINCE, which stands for *Projects in Controlled Environments*, is a project management method covering the organization, management and control of projects.

Agile Modelling and RUP

- The Unified Process or the Rational Unified Process (sometimes known as RUP) is a framework for handling the whole lifecycle of a software development project. It is referred to as a framework because it allows the overall suite of activities to be customized and modified as required for a particular type of application . this is illustrated in Figure 11.1

Why then is the Unified Process called a *process* and not the Unified Framework?

- It is called a process because its primary aim is to define:
- Who is doing what?
- When they do it?
- How to reach a certain goal (i.e. each activity)?
- The inputs and outputs of each activity.

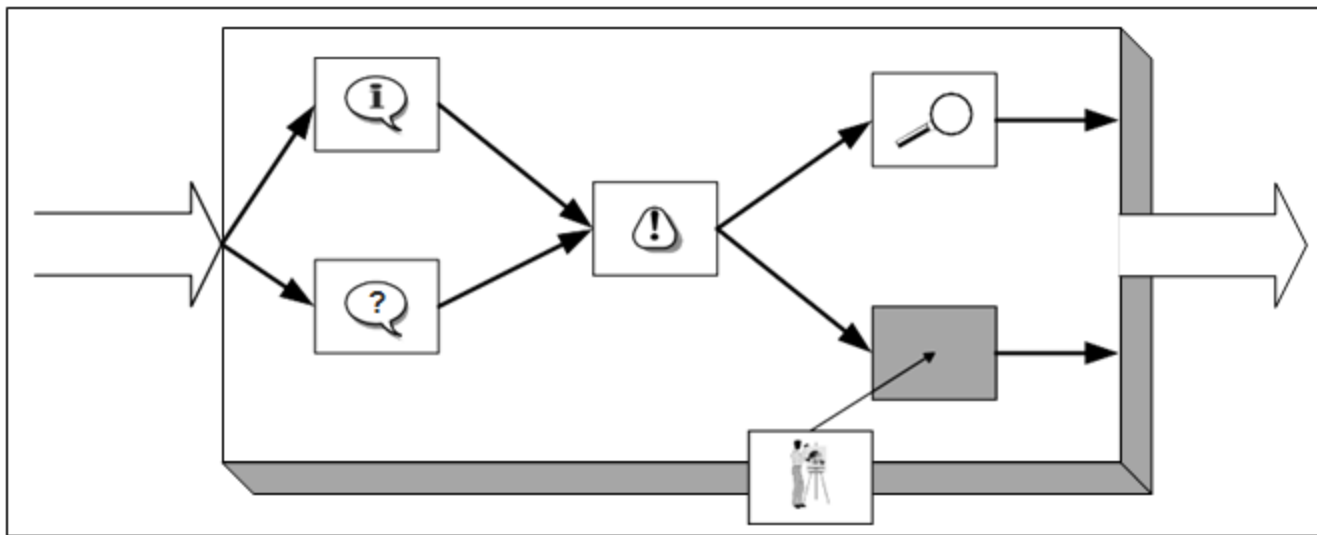


Fig. 11.1 The Unified Process is a framework.

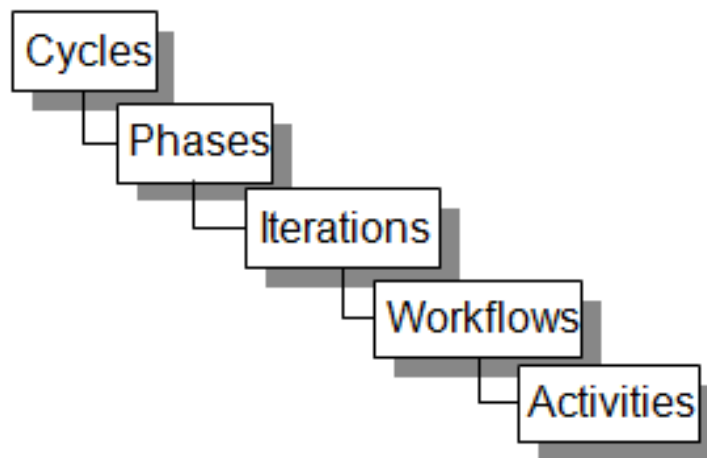


Fig. 11.2 Key building blocks of the Unified Process.

- The Unified Process is actually comprised of low-level activities (such as finding classes), which are combined together into disciplines formally known as work- flows (which describe how one activity feeds into another).
- These disciplines are organized into iterations. Each iteration identifies some aspect of the system to be considered. How this is done is considered in more detail later. Iterations themselves are organized into phases.
- Phases focus on different aspects of the design process, for example, requirements, analysis, design and implementation. In turn, phases can be grouped into cycles. Cycles focus on the generation of successive releases of a system (for example, version 1.0, version 1.1, etc.).

Overview of the Unified Process

- There are four key elements to the philosophy behind the Unified Process. These four elements are:
- Iterative and incremental,
- Use-case driven,
- Architecture-centric,
- Acknowledges risk.

Iterative and Incremental

The Unified Process is iterative and incremental, that is, the design process is based on iterations which address either different aspects of the design process or move the design forward in some way (this is the incremental aspect of the model). This does not mean that the Unified Process is a process based on rapid prototyping. Any prototypes that are developed in the Unified Process are used to explore some aspect of the design.

Essentially, the following holds with the iterative approach in the Unified Process:

- You plan a little.
- You specify, design and implement a little.
- You integrate, test and run.
- You obtain feedback before the next iteration.

The end result is that you incrementally produce the system being designed. While you do this you explicitly identify the risks to your design/system Unified Process front and deal with them early on (see later).

Use-Case Driven

- The Unified Process is also use-case driven. Remember from earlier that use-cases help identify who uses the system and what they need to do with the system (i.e., the top-level functionality). Thus, use-cases help identify the primary requirements of the system. One problem with many traditional approaches is that once the requirements have been identified there is no traceability of those requirements through the design to the implementation.
- In the Unified Process, use-cases are used to ensure that the evolving design is always relevant to what the user required. Indeed, the use cases act as the one consistent thread throughout the whole of the development process as illustrated in Figure 11.3.
- For example, at the beginning of the design phase, one of the two primary inputs to this phase is the use-case model. Then explicitly within the design model are use-case realizations that illustrate how each use-case is supported by the design

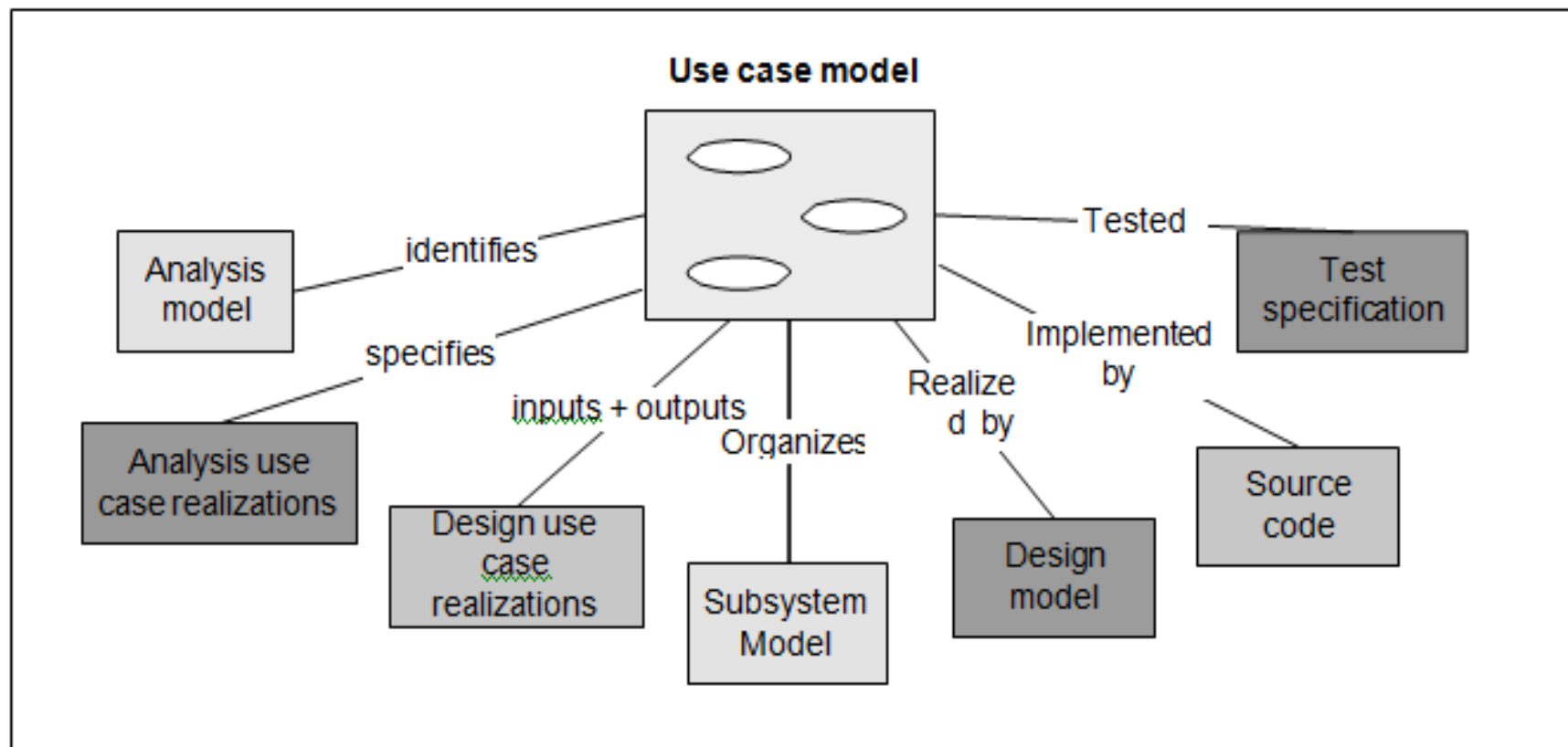


Fig. 11.3 The role of use-cases.

To summaries the role of use-cases they:

- Identify the users of the system and their requirements.
- Aid in the creation and validation of the system's architecture.
- Help produce the definition of test cases and procedures.
- Direct the planning of iterations.
- Drive the creation of user documentation.
- Direct the deployment of system.
- Synchronies the content of different models.
- Drive traceability throughout models.

Lifecycle Phases

The Unified Process is comprised of four distinct phases. These four phases (presented in Figure 11.4) focus on different aspects of the design process. The four phases are Inception, Elaboration, Construction and Transition.

The four phases and their roles are outlined below.

- *Inception*. This phase defines the scope of the project and develops the business case for the system. It also establishes the feasibility of the system to be built. Various prototypes may be developed during this phase to ensure the feasibility of the proposal

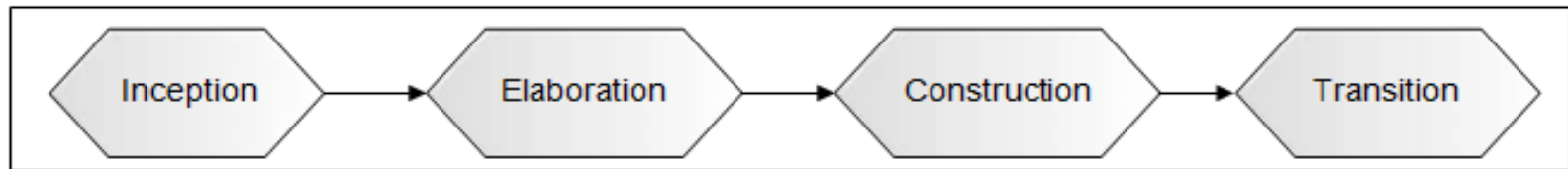


Fig. 11.4 Four phases of the Unified Process. |

- *Elaboration*. This phase captures the functional requirements of the system. It should also specify any non-functional requirements to ensure that they are taken into account. The other primary task for this phase is the creation of the architecture to be used throughout the remainder of the Unified Process.
- *Construction*. This phase concentrates on completing the analysis of the system, performing the majority of the design and the implementation of the system. That is, it essentially builds the product.
- *Transition*. The transition phase moves the system into the users environment.

This involves activities such as deploying the system and maintaining it.

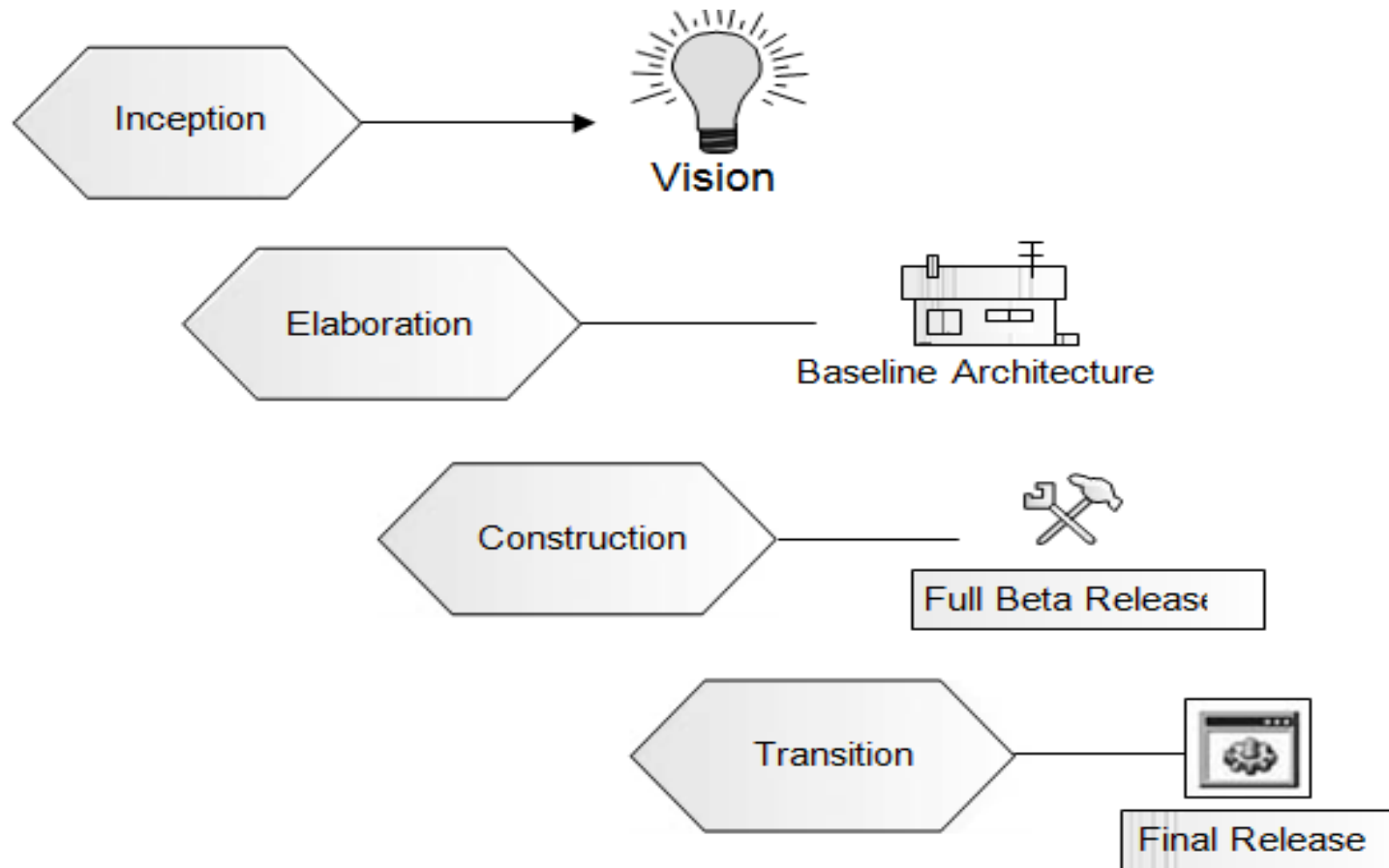


Fig. 11.5 Major deliverables of each phase.

Each phase has a set of major milestones that are used to judge the progress of the overall Unified Process (of course, with each phase there are numerous minor milestones to be achieved). The primary milestones (or products) of the four phases are illustrated in Figure 11.5.

A milestone is the culmination of a phase and is comprised of a set of artifacts (such as specific models) that are the product of the disciplines (and thus activities) in that phase. The primary milestones for each phase are:

- *Inception*. The output of this phase is the vision for the system. This includes a very simplified use-case model (to identify what the primary functionality of the system is), a very tentative architecture and the most important or significant risks are identified and the elaboration phase is planned.
- *Elaboration*. The primary output of this phase is the architecture along with a detailed use-case model and a set of plans for the construction phase.
- *Construction*. The end result of this phase is the implemented product that includes the software as well as the design and associated models. The product may not be without defects as some further work has yet to be completed in the transition phase.
- *Transition*. The transition phase is the last phase of a cycle. The major milestone met by this phase is the final production quality release of the system.

Phases, Iterations and Disciplines

- There can be confusion over the relationship between phases and disciplines. Not least because a single discipline can cross (or be involved in) more than one phase (see Figure 11.6). The various phases highlight the satisfaction of these milestones. For example, during the Elaboration phase, part of the requirements, analysis, design and even implementation disciplines may act.
- The five disciplines in the Unified Process are Requirements, Analysis, Design, Implementation and Test (as indicated in Figure 11.6).
- Note that the Design, Implementation and Test disciplines are broken down in the Unified Process. This is to indicate that elements of each discipline may take place earlier than the core parts of the discipline

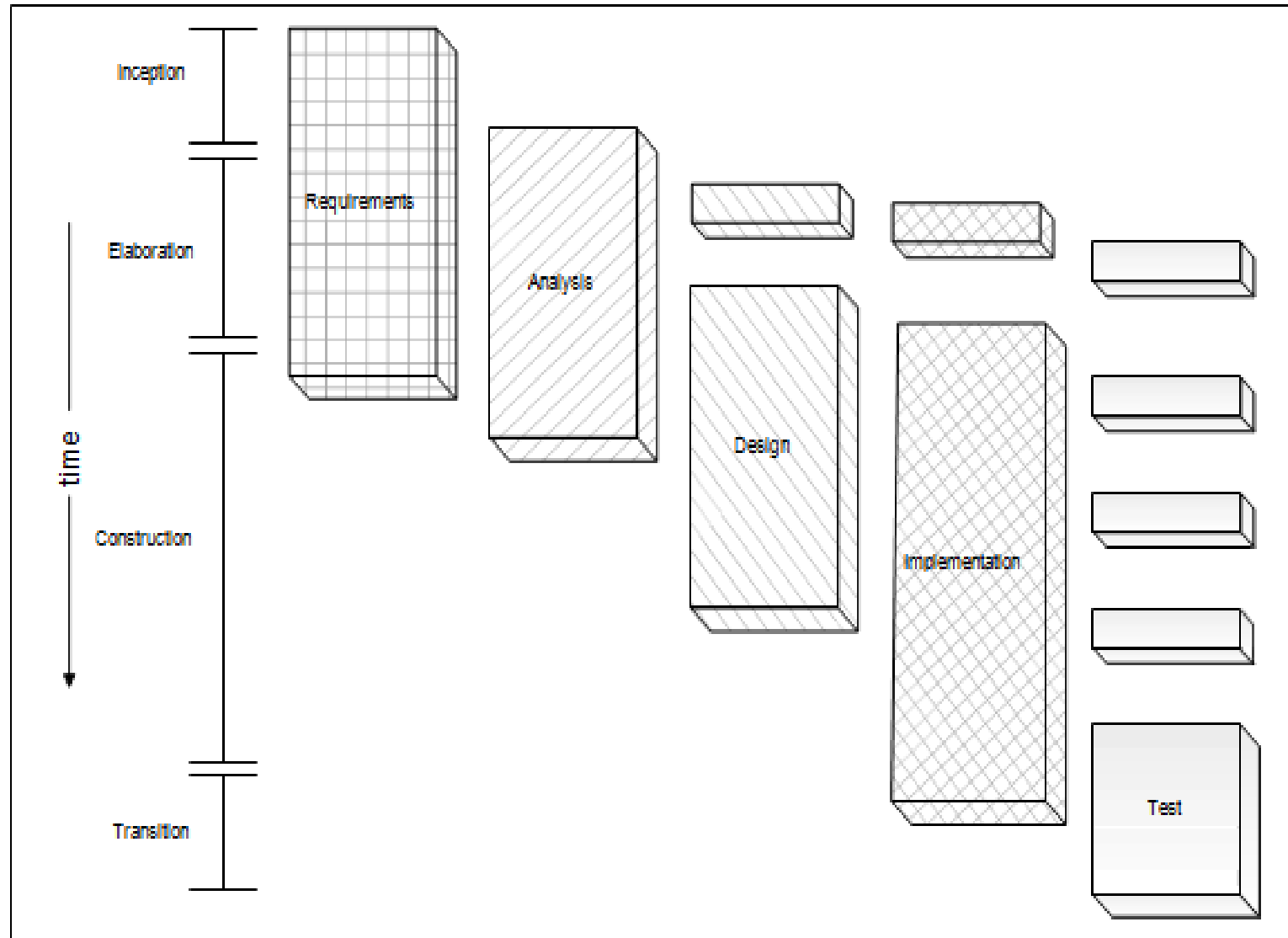


Fig. 11.6 Disciplines versus phases.

The focus of each discipline is described below (their primary products are illustrated in Figure 11.7).

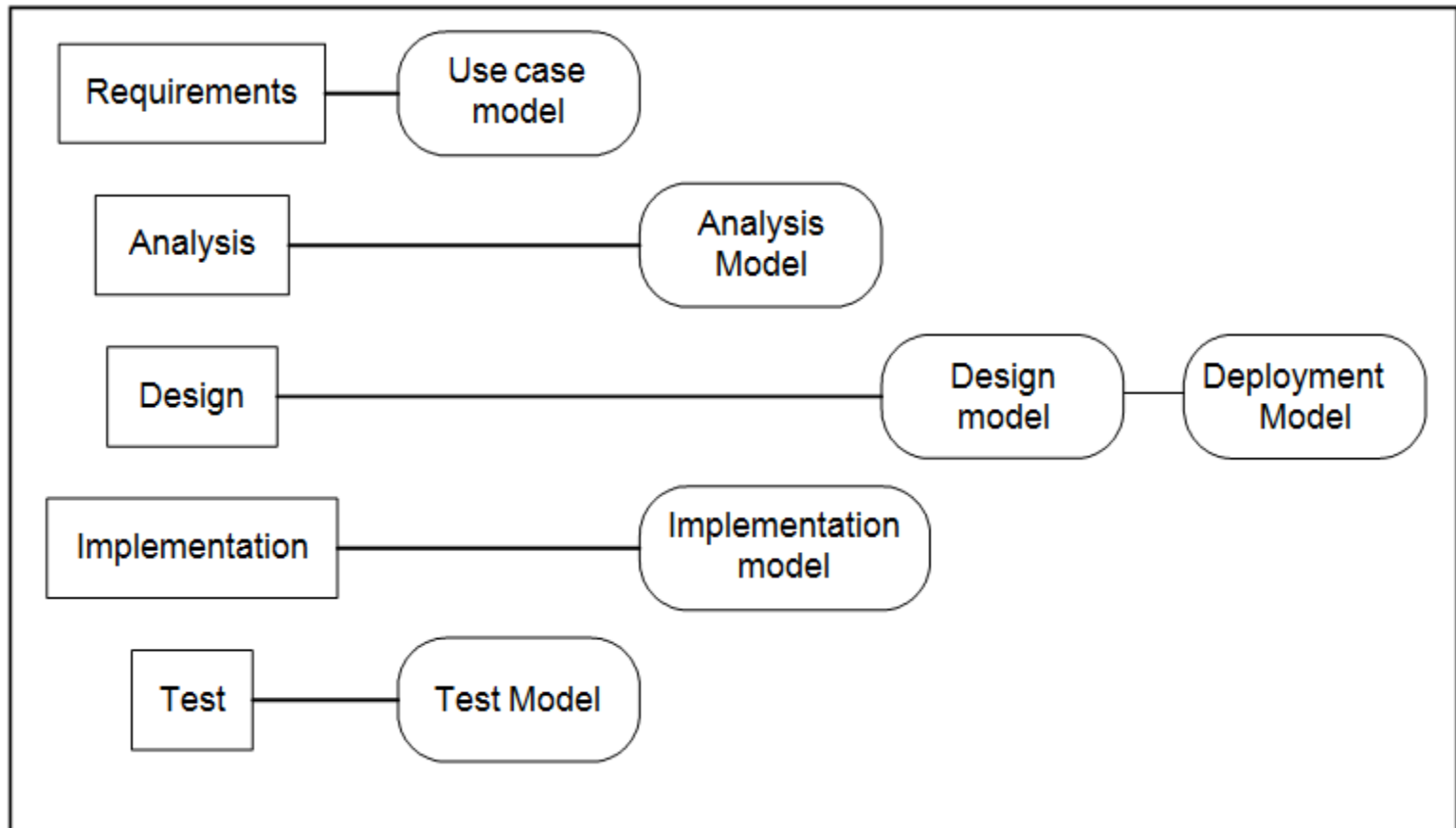


Fig.11.7 Discipline products.

- *Requirements.* This discipline focuses on the activities that allow the functional and non-functional requirements of the system to be identified. The primary product of this discipline is the use-case model.
- *Analysis.* The aim of this discipline is to restructure the requirements identified in the requirements discipline in terms of the software to be built rather than in the users less precise terms. It can be seen as a first cut at a design; however, that is to miss the point of what this discipline aims to achieve.
- *Design.* The design discipline produces the detailed design that will be implemented in the next discipline.
- *Implementation.* This discipline represents the coding of the design in an appropriate programming language (for this book that is Java), the compilation, packaging, deployment and documentation of the software.
- *Test.* The test discipline describes the activities to be carried out to test the software to ensure that it meets the users requirements, that it is reliable, etc.

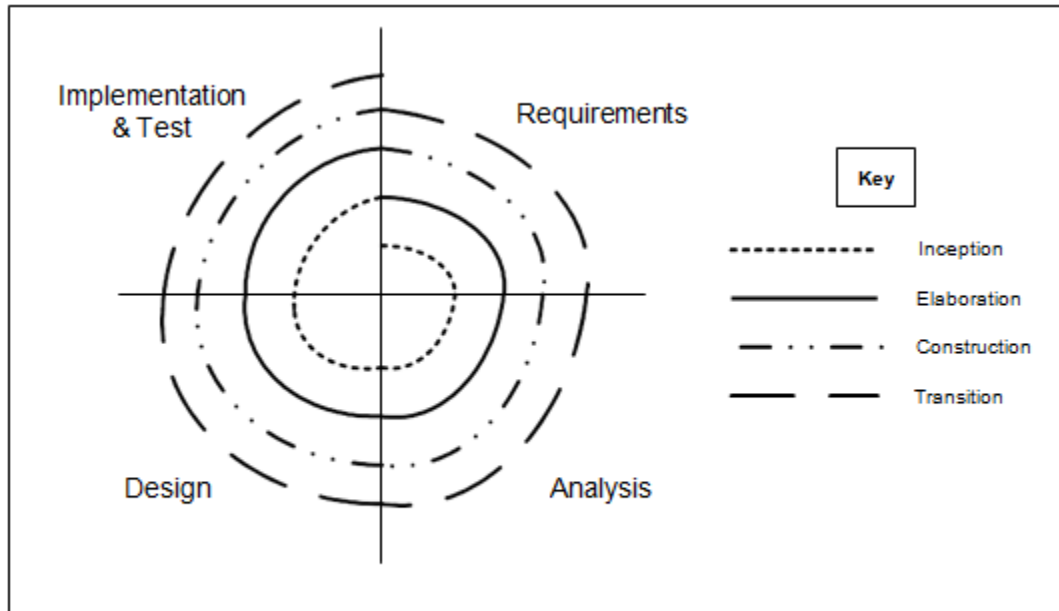


Fig. 11.8 The Unified Process is a spiral.

The Unified Process is a spiral (as indicated by its iterative and incremental nature). This is illustrated in Figure 11.8 (note as a phase moves around the spiral multiple iterations may occur;

Each phases moves around the various disciplines producing outputs that feed into the next phase. Each phase examines the requirements (to a greater or lesser extent). Each phase involves the analysis discipline, the design discipline and so on

Modeling and the Unified Process

- It is a framework that encourages you to adapt it to your own needs.
- The Unified Process actively encourages an iterative and incremental approach to software development (as illustrated in Figure 11.9). Indeed, it is one of the key elements of Unified Process.
- There is nothing in the Unified Process that assumes that *all* the modeling must be done upfront. Indeed, if you adopt an incremental and iterative style, then each iteration will generate its own set of models.

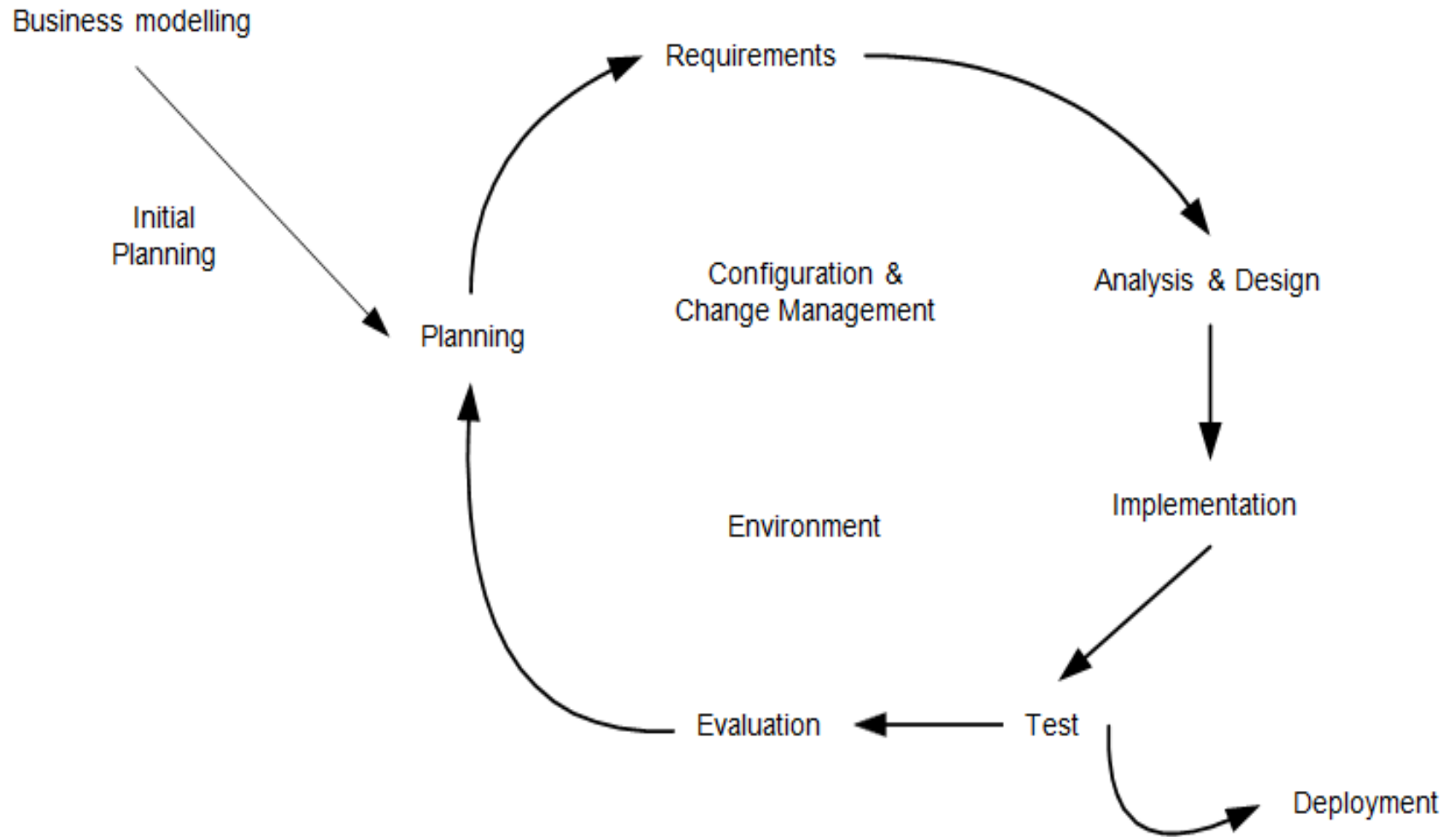


Fig. 11.9 The Unified Process iterative and incremental.

FDD and RUP

- Agile Modeling is not the only agile methodology that can be usefully applied to a software project that is employing the Unified Process
- Feature-centric means that each iteration centers on the identification and realization of system features. A feature is a schedulable requirement associated with the activity used to realize it. These requirements may be user-related requirements, application behavior requirements or internal requirements. The features can then be grouped into work packages that can act as the basis of the planning required to monitor and manage the software development process.

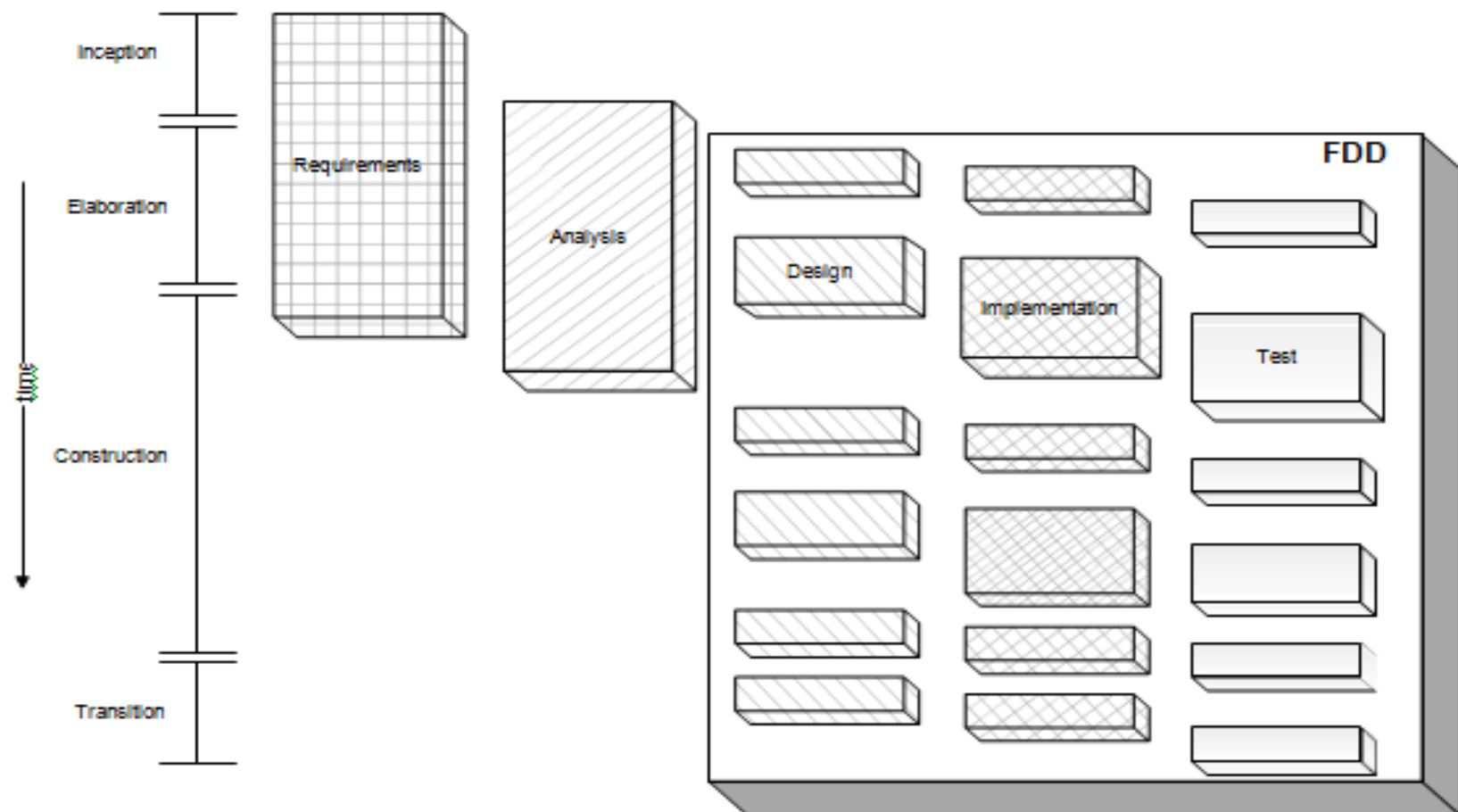


Fig. 11.10 Integrating Feature-Driven Development with the Unified Process.

- For example, earlier it was stated that the iterative approach within the
- Unified Process essentially has the following steps (which actually look remarkably oriented towards the agile movement anyway):
- You plan a little.
- You specify, design and implement a little.
- You integrate, test and run.
- You obtain feedback before next iteration.
- The Feature-Driven Development process therefore essentially handles these steps. This is illustrated in Figure 11.10 where the design, implementation and test disciplines have been subsumed by the Feature-Driven Development process. Thus, we do not have a big up-front design, followed (possibly in parallel) by the implementation and then the testing. Rather we have iterations comprising planning, design, implementation and testing in an incremental and iterative fashion. Indeed, the typical approach is to have multiple tasks during which a bit of design, implementation and testing occur. To indicate that the iterations managed by the FDD process may be of different sizes, the boxes being subsumed by the FDD process are presented in different sizes

Agile Methods and Prince2

- The PRINCE2 method describes how a project is divided into manageable stages enabling efficient control of resources and regular progress monitoring through- out the project.
- Project planning using PRINCE2 is product-based, which means the project plans are focused on delivering results and are not simply about planning when the various activities on the project will be done.
- Because an iterative or incremental approach may at first sight appear less controlled than an approach such as PRINCE2, some have perceived that this means that PRINCE2 and such approaches are inconsistent. However, this is not the case. Indeed, if you consider the emphasis of the Feature-Driven approach, then similarities with some aspects of PRINCE2 can immediately be seen. For example, product (or feature)-based planning, the involved partnership of users and developers and the strong emphasis on the underlying business need (or case).

Table 11.1 makes it clear that the iterative approach includes some project management content while in turn PRINCE2 maintains a view of the whole project (whereas the iterative approach plans out each iteration at the start of that iteration). In addition, the iterative approach fixes the length of an iteration in terms of time and then attempts to achieve a prioritized list of features. Some features may not be implemented in the time available and will “return to the pot” for the next iteration.

Table 11.1 PRINCE2 and FDD relationships.

	<i>PRINCE2</i>	<i>Iterative/Feature-Driven Design</i>
Organization	Project board and assurance	Project manager
Project structure	Stages	Time boxed increments/iterations based on features
Outputs	Management products	Iterative releases
Quality	Quality plan	Test specification, plan and report per iteration
Flexibility/change	Time, cost	Features
Control	Issue management	Time boxing, feature selection and management

Products produced as part of the PRINCE2 process are *management* and *quality* products. They relate to the effective and efficient management and control of the project and to project quality, respectively. Most products within the Feature-Driven approach are *specialist* products, that is, they are either descriptions of features, or descriptions of how the system operates with these features (use-case documents), or the techniques to be used.

- To avoid duplication of effort, the recommended approach is that high-level management and quality products should be the province of PRINCE2 and that detailed iteration planning and quality monitoring should be the province of the iterative approach.
- Managing and controlling an iterative project using PRINCE2 is fundamentally the same as for any other PRINCE2 project. The purpose is to enable each level of the project management team to:
- Demonstrate to the next level up that the project is on track to a successful outcome (that the project will deliver products that are fit for business purpose on time and within budget). Identify early anything that may prevent this.

In PRINCE2, management and control are done at each project stage and everything depends on how the project is broken up into stages. PRINCE2 defines major control points through the life of the project, as follows:

- Project initiation,
 - End stage assessment,
- Regular highlight reports,
- Exception reports,
- Mid-stage assessment,
- Project closure.
- The results from each iteration can feedback into the monitoring and reporting stages of PRINCE2.

Questions on UNIT-V

- 1.How can Agile Modelling be integrated with RUP?
- 2.What are the similarities between Agile Modelling and RUP's iterative nature?
- 3.What are the four key elements to the philosophy behind the Unified Process ?
- 4.How is Unified Process is iterative and incremental ?
- 5.How does Unified Process is also use-case driven and use-cases help identify who uses the system and the top-level functionality.
- 6.What are the four phases of Unified Process ,their roles and their outputs.
- 7.List out the relationship between phases and disciplines.
- 8.How does Modelling align with the Unified Process.
- 9.How does Feature-Driven Development align with RUP?

10. What is PRINCE2, and how does it differ from Agile methods?
11. Explain the Differences between PRINCE2 and Iterative /Feature-driven Design .
12. Name popular Agile tools and explain their roles (e.g., JIRA, Trello, Git, Jenkins).
13. What are the characteristics of a good Agile tool?
14. What is Eclipse and why is it considered suitable for Agile development?
15. Name popular Agile tools and explain their roles
 1. Omondo's modeling plug-in for Eclipse,
 2. ANT: the build tool for Java,
 3. The CVS version control system,
 4. The JUnit Java test framework.
16. What are some key plugins in Eclipse that enhance Agile productivity?